

나만의 Streamlit 웹 애플리케이션

02 Streamlit과 함께하는 데이터 분석

목차

—
Streamlit을 통한 여러
가지 시각화 웹 앱 제작

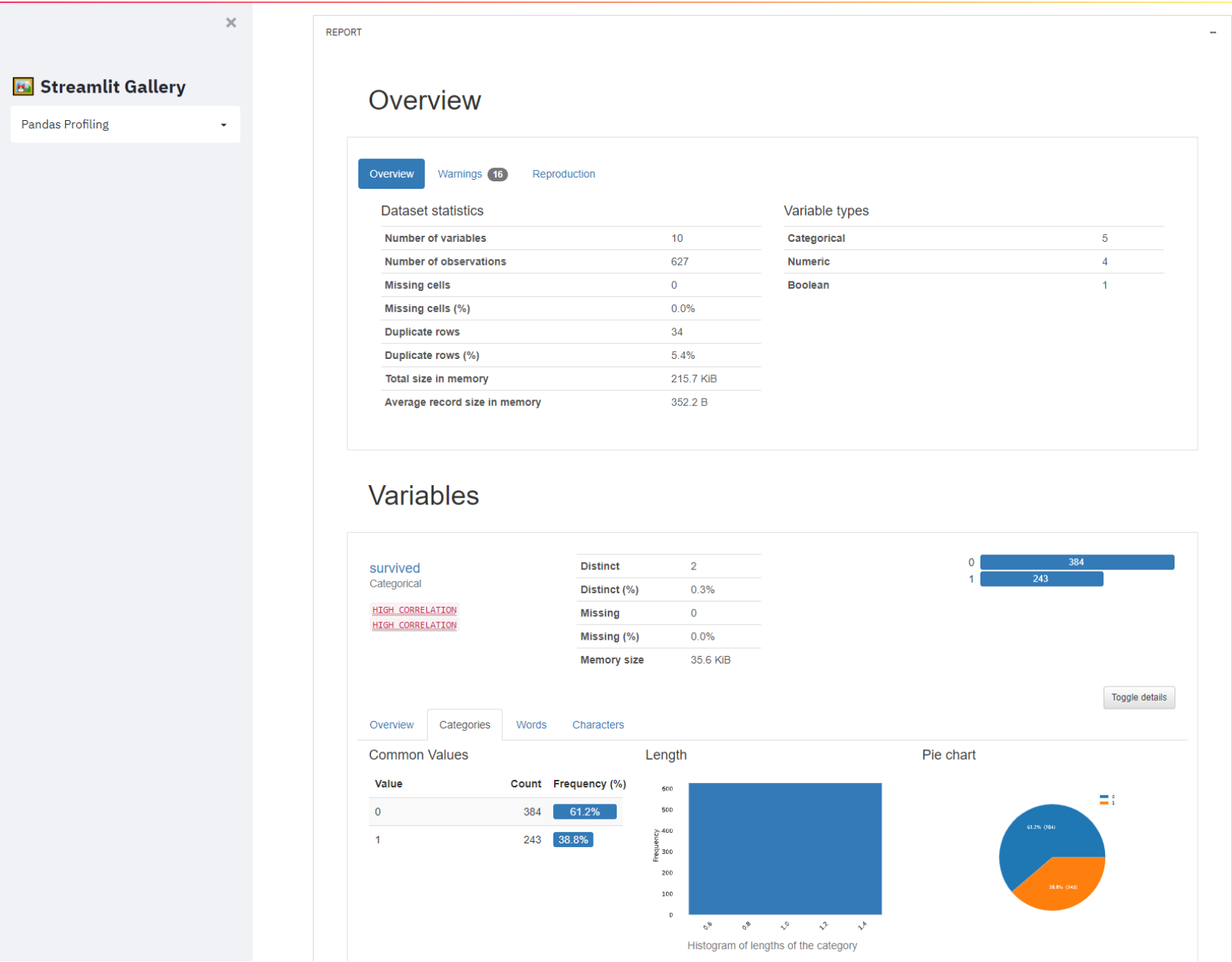
- 01 데이터 분석을 위한 Elements, Widgets
- 02 [실습] Data Elements와 여러가지 테이블
- 03 [실습] Chart Elements와 여러가지 시각화
- 04 [실습] 지도 그리기

01

데이터 분석을 위한 Elements, Widgets

01 데이터 분석을 위한 Elements, Widgets

✔ Streamlit으로 하는 데이터 분석



데이터 분석과 머신러닝을 지원해주는 툴로
데이터 분석 웹 애플리케이션 기능 보유

01 데이터 분석을 위한 Elements, Widgets

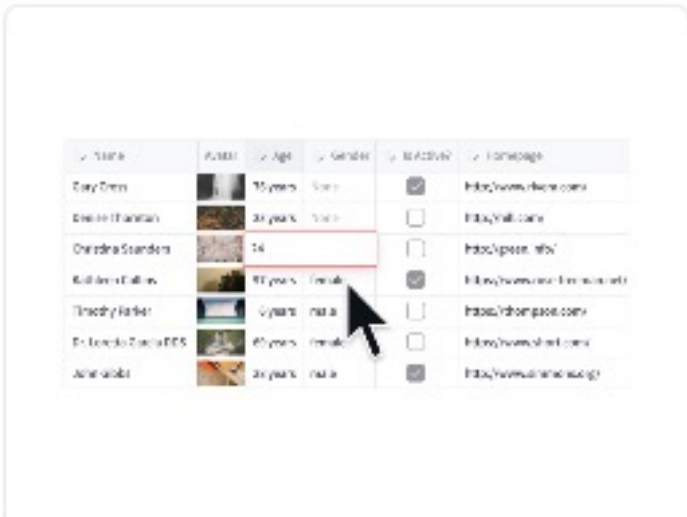
☑ Streamlit 데이터 분석을 위한 Data Elements



Dataframes

Display a dataframe as an interactive table.

```
st.dataframe(my_data_frame)
```



Data editor

Display a data editor widget.

```
edited = st.data_editor(df, num
```



Column configuration

Configure the display and editing behavior of dataframes and data editors.

```
st.column_config.NumberColumn
```



Metrics

Display a metric in big bold font, with an optional indicator of how the metric changed.

```
st.metric("My metric", 42, 2)
```


01 데이터 분석을 위한 Elements, Widgets

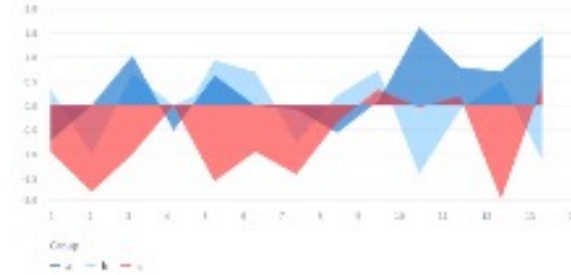
☑ Streamlit 데이터 분석을 위한 Chart Elements



Simple line charts

Display a line chart.

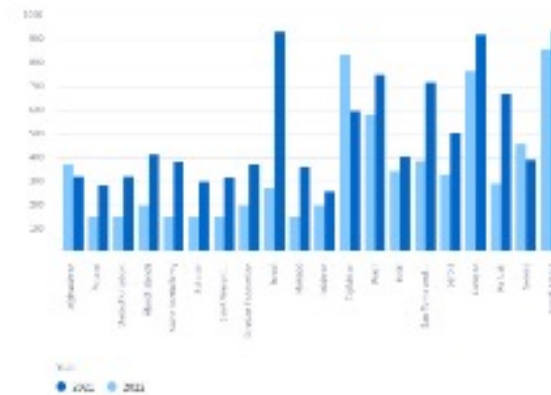
```
st.line_chart(my_data_frame)
```



Simple area charts

Display an area chart.

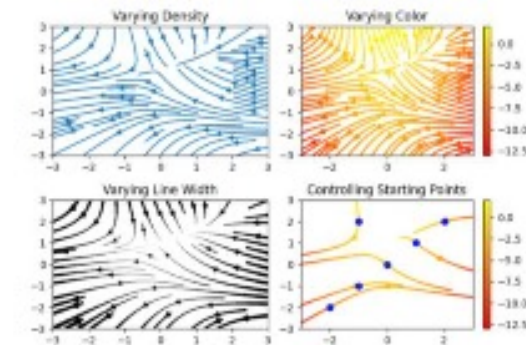
```
st.area_chart(my_data_frame)
```



Simple bar charts

Display a bar chart.

```
st.bar_chart(my_data_frame)
```



Matplotlib

Display a matplotlib.pyplot figure.

```
st.pyplot(my_mpl_figure)
```



Streamlit Folium

Streamlit Component for rendering Folium maps. Created by [@randyztwitch](#).

```
m = folium.Map(location=[39.94  
st_data = st_folium(m, width=7
```


01 데이터 분석을 위한 Elements, Widgets

☑ Data Elements - st.dataframe

```
import streamlit as st
import pandas as pd
import numpy as np

df = pd.DataFrame(
    np.random.randn(50, 20),
    columns=('col %d' % i for i in range(20)))

st.dataframe(df) # Same as st.write(df)
```

	col 0	col 1	col 2	col 3	col 4	col 5	col 6	col 7	col 8	col 9	col 10
0	-0.1894	0.4283	0.4087	0.3539	0.881	0.392	0.0642	0.0479	0.0957	-0.0703	-0.7732
1	0.5224	0.6675	1.1465	0.4144	0.0334	-0.9711	1.0781	-0.9396	0.1072	-0.0496	-0.2818
2	-0.6847	-0.653	0.15	-0.3329	0.6689	0.2793	-1.3614	1.3484	0.412	0.6549	1.4022
3	-0.3557	-2.3596	-0.8624	0.8539	0.1074	0.7319	-0.1469	0.7568	-0.9802	0.861	0.9945
4	1.5941	0.3398	0.9615	0.8168	-0.8505	0.4503	0.9793	-0.3308	-1.4387	0.6949	0.9555
5	0.4858	0.9	-0.9217	-0.7718	0.404	-0.4346	-0.8378	0.0993	-1.1775	-0.5639	1.8438
6	0.5737	0.1769	-0.6096	2.6857	1.165	-1.1585	1.7803	-0.1448	-1.2124	-1.3247	-1.3514
7	-0.2624	0.7143	2.3	0.7281	-0.7129	-0.416	-1.6043	-0.8089	-0.1224	-1.1428	2.0306
8	0.2879	-0.1469	1.0615	0.2871	0.5955	-1.7389	-1.1996	1.3133	-1.3951	1.6542	-0.079

Pandas의 데이터 프레임을 렌더링

데이터의 원본 출력 및 확인

출력한 결과 테이블은 사용자와 상호작용 가능

01 데이터 분석을 위한 Elements, Widgets

☑ Data Elements - st.data_editor

```
import streamlit as st
import pandas as pd

df = pd.DataFrame([
    {"command": "st.selectbox", "rating": 4, "is_widget": True},
    {"command": "st.balloons", "rating": 5, "is_widget": False},
    {"command": "st.time_input", "rating": 3, "is_widget": True},
])

edited_df = st.data_editor(df)

favorite_command = edited_df.loc[edited_df["rating"].idxmax()]["command"]
st.markdown(f"Your favorite command is **{favorite_command}** 🍷")
```

	⌵ command	⌵ rating	⌵ is_widget
0	st.selectbox	6	☒
1	st.balloons	5	☐
2	st.time_input	3	☒

Your favorite command is st.selectbox 🍷

표에서 데이터 수정 가능

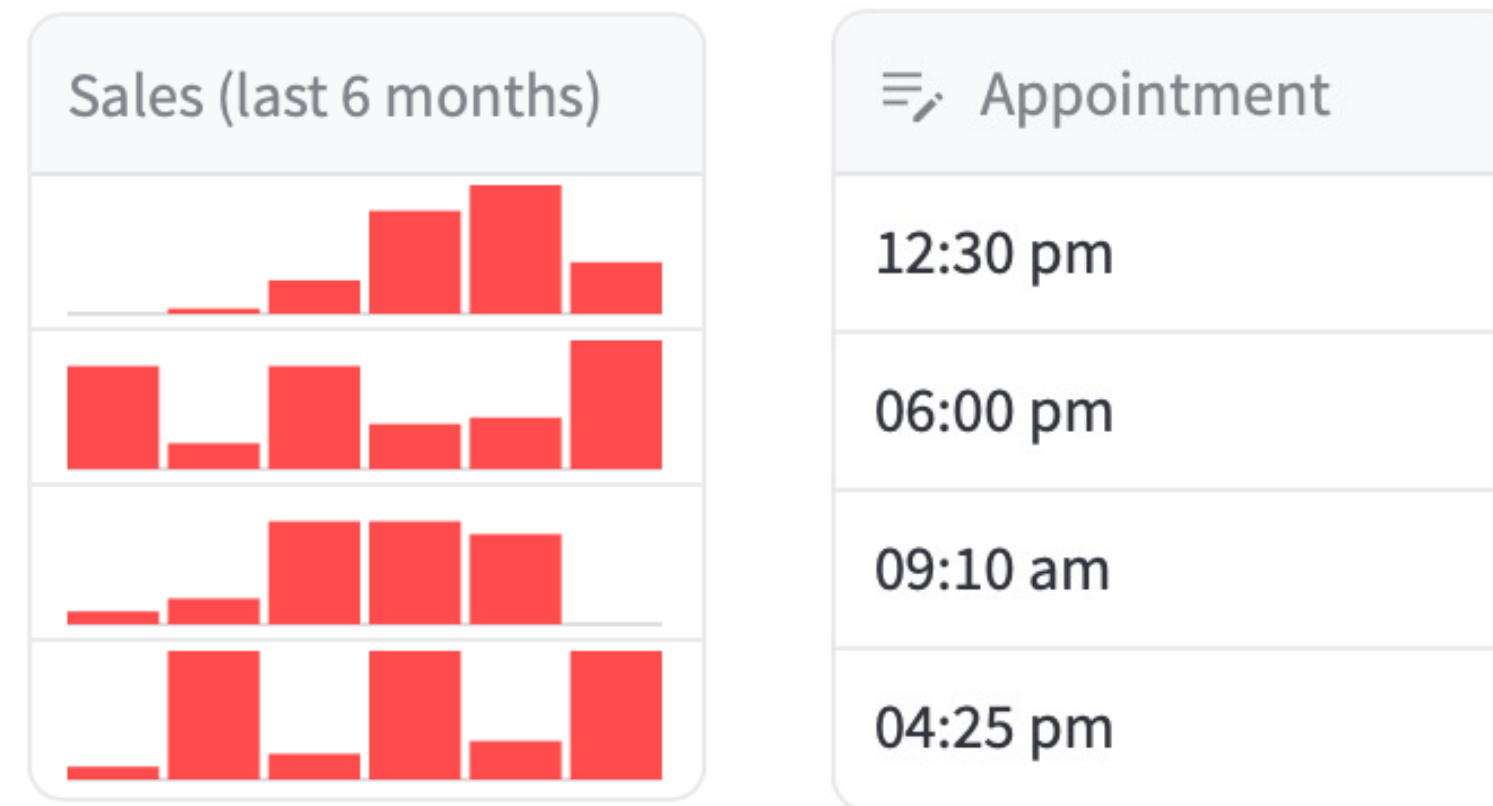
수정 결과에 따라 페이지의 다른 내용도 동적으로 반응

사용자와 반응하는 표 삽입

01 데이터 분석을 위한 Elements, Widgets

☑ Data Elements - st.column_config

```
st.data_editor(  
    data_df,  
    column_config={  
        "sales": st.column_config.BarChartColumn(  
            "Sales (last 6 months)",  
            help="The sales volume in the last 6 months",  
            y_min=0,  
            y_max=100,  
        ),  
    },  
    hide_index=True,  
)
```



테이블 안에
그래프, 체크박스, 이미지 등을
넣을 수 있도록 돕는 기능



보다 **다양한** 표를 넣을 수 있다!

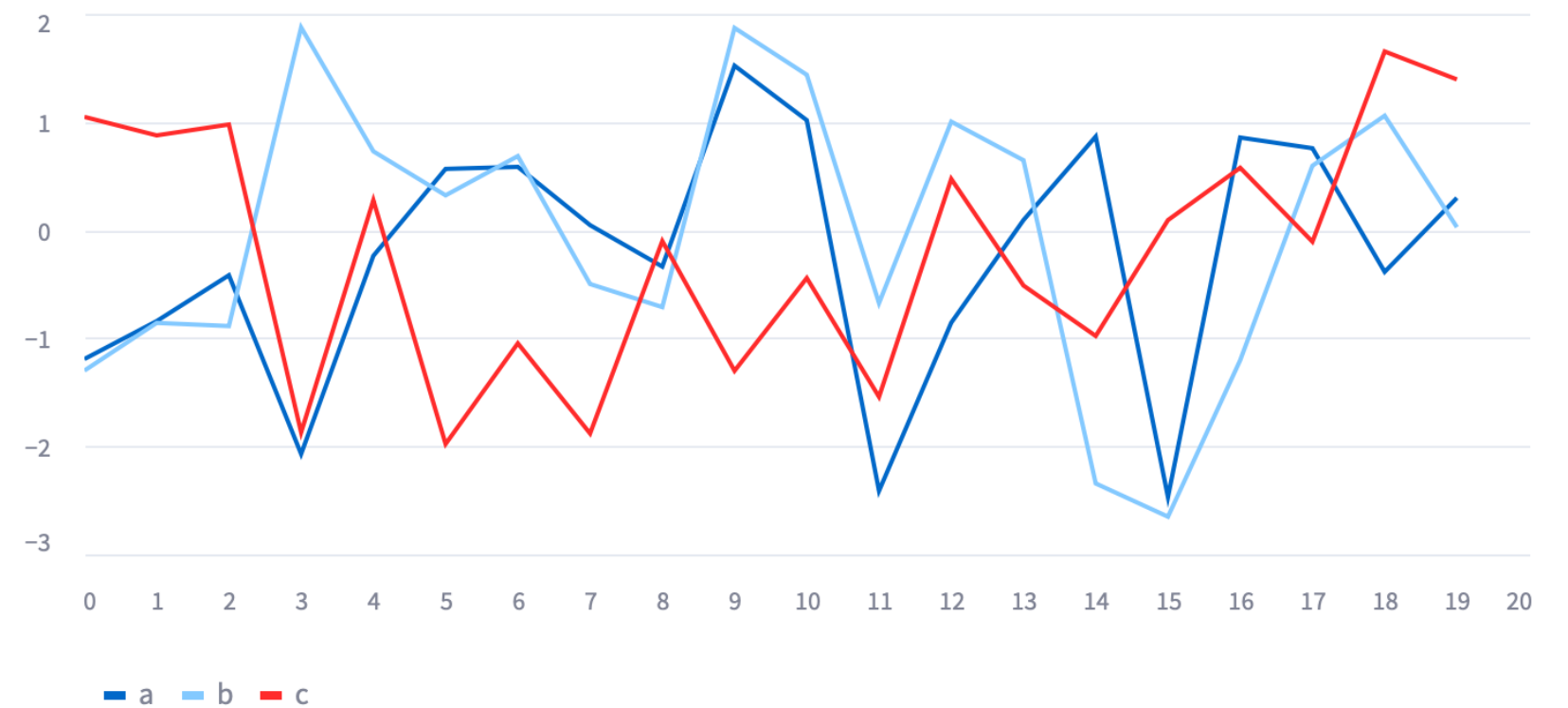
01 데이터 분석을 위한 Elements, Widgets

☑ Chart Elements - Line charts

```
import streamlit as st
import pandas as pd
import numpy as np

chart_data = pd.DataFrame(
    np.random.randn(20, 3),
    columns=['a', 'b', 'c'])

st.line_chart(chart_data)
```



선 그래프를 화면에 출력

연속적으로 변하는 데이터를 한눈에 확인 가능

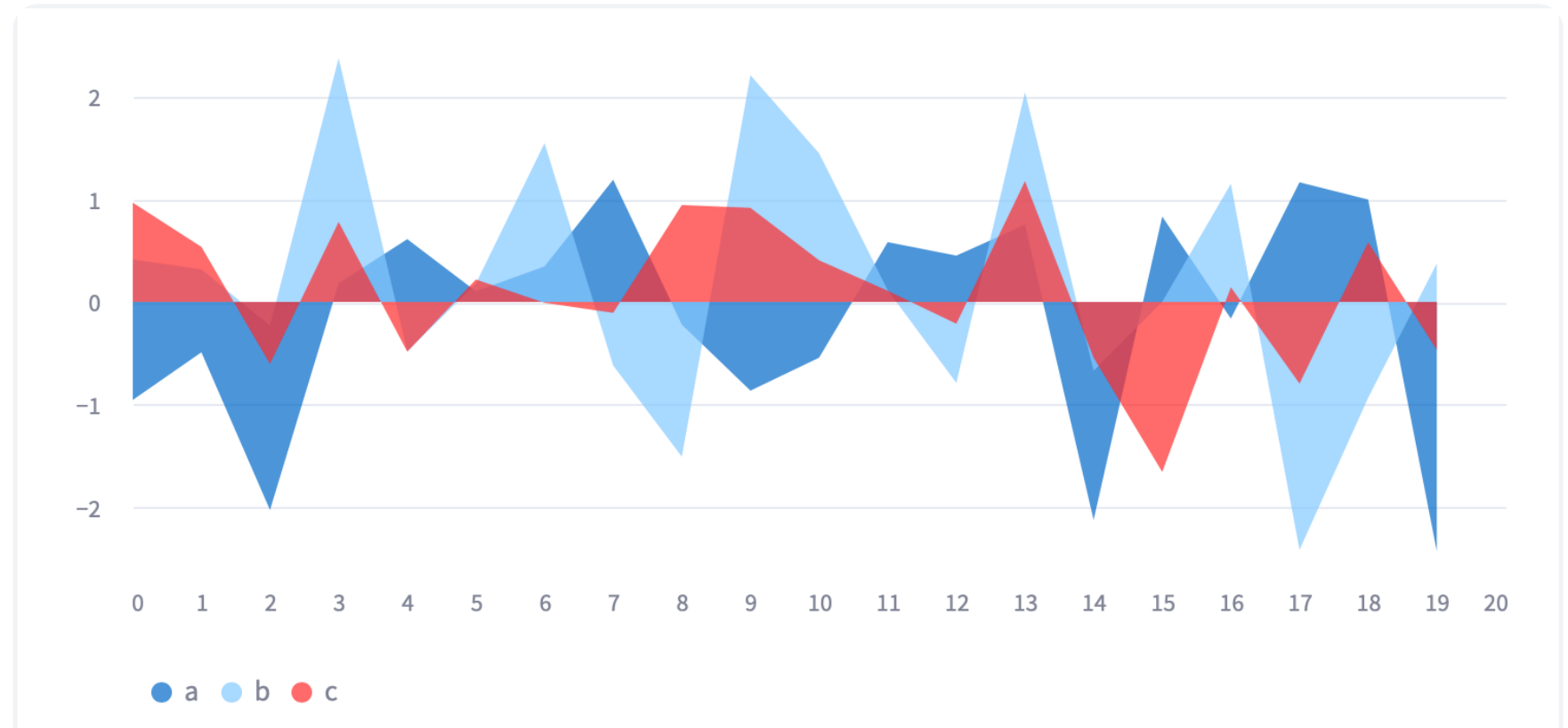
01 데이터 분석을 위한 Elements, Widgets

☑ Chart Elements - Area charts

```
import streamlit as st
import pandas as pd
import numpy as np

chart_data = pd.DataFrame(
    np.random.randn(20, 3),
    columns=['a', 'b', 'c'])

st.area_chart(chart_data)
```



영역 그래프를 화면에 출력

각 데이터가 차지하는 부분을 면적으로서 확인 가능

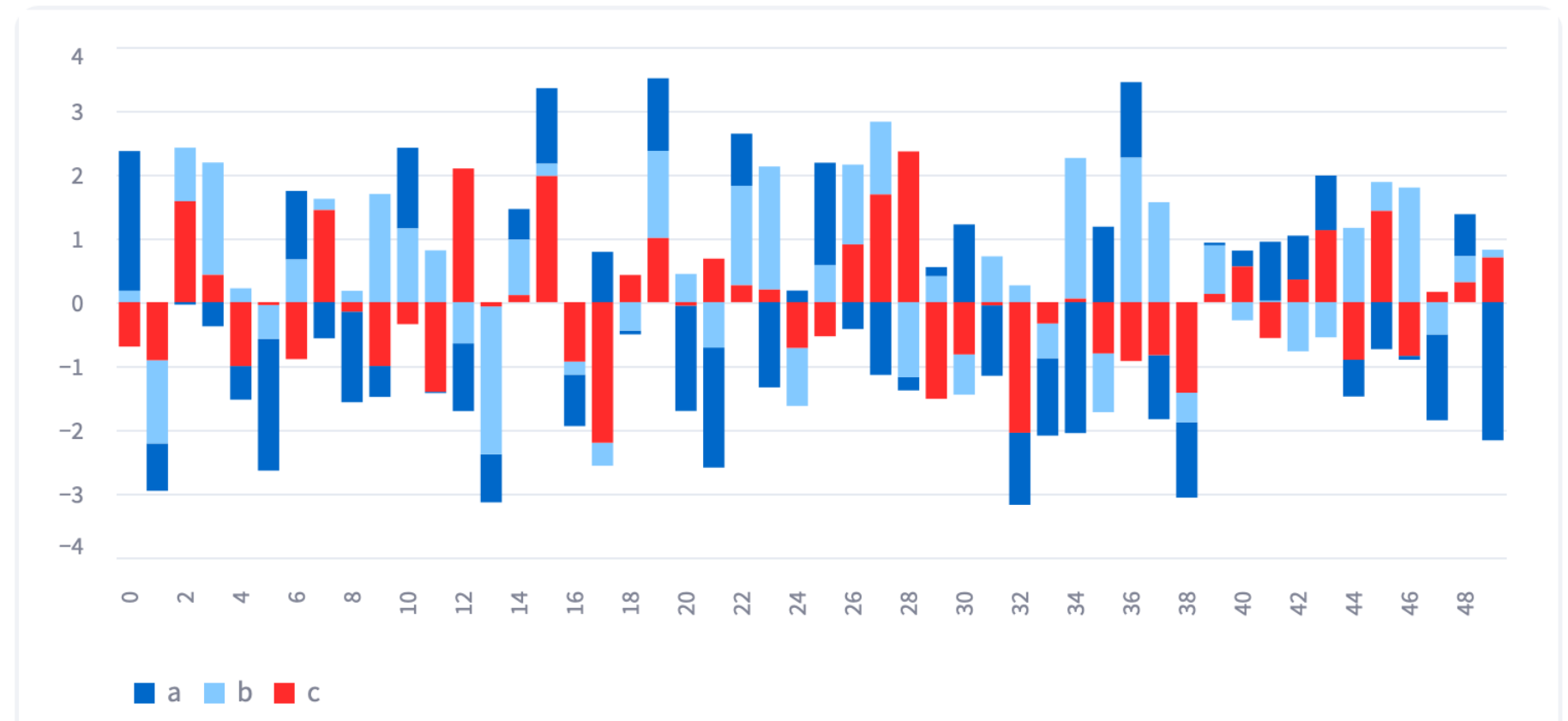
01 데이터 분석을 위한 Elements, Widgets

☑ Chart Elements - Bar charts

```
import streamlit as st
import pandas as pd
import numpy as np

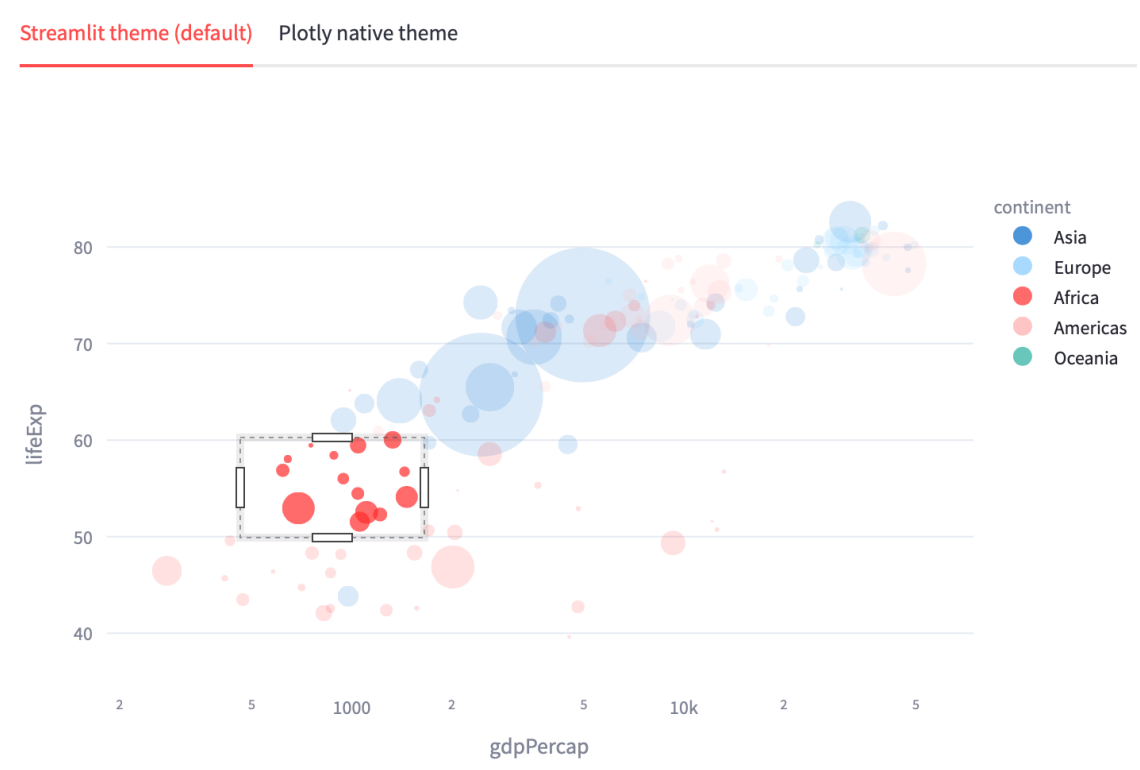
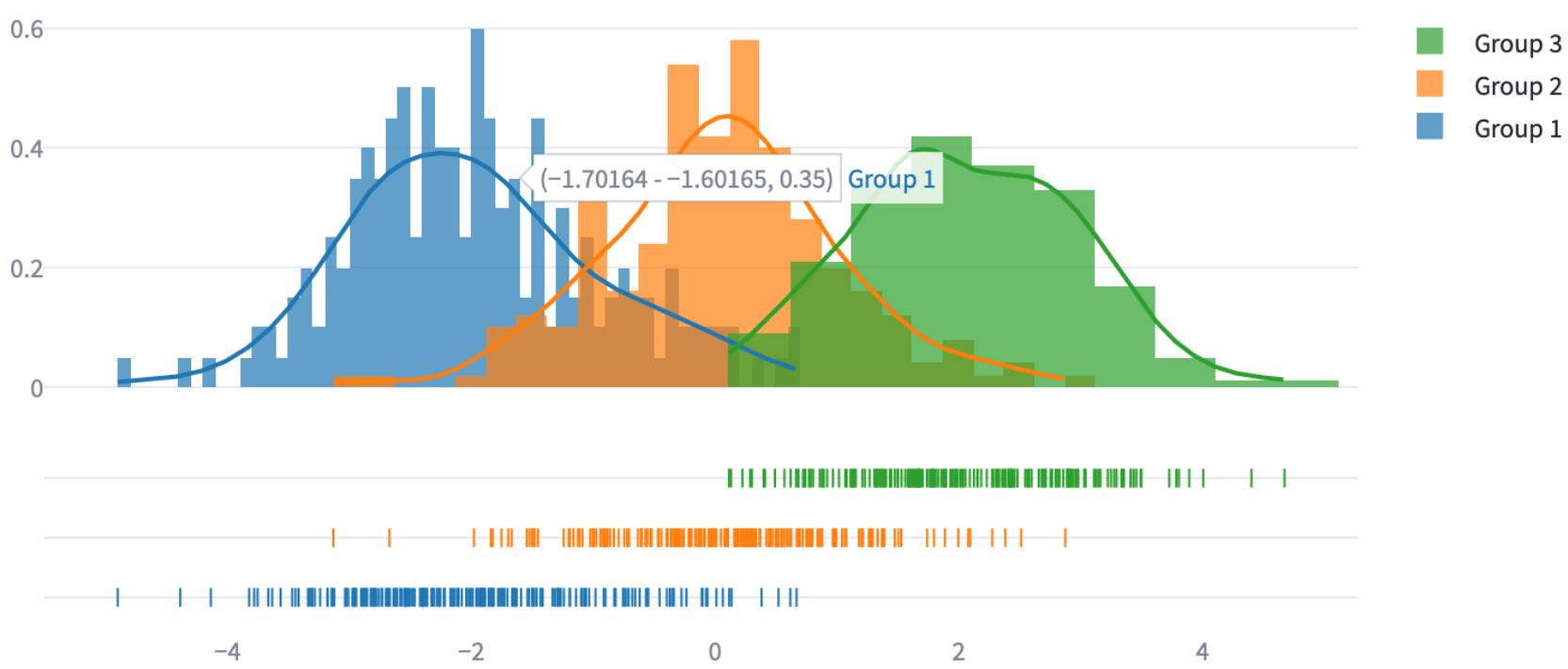
chart_data = pd.DataFrame(
    np.random.randn(20, 3),
    columns=["a", "b", "c"])

st.bar_chart(chart_data)
```



막대 그래프를 화면에 출력
데이터의 각 범주가 갖는 양을 손 쉽게 비교

Chart Elements - Streamlit x Plotly



Plotly 시각화 라이브러리 그래프 랜더링

용어 해설

Plotly

인터랙티브한 그래프를 만들어주는
파이썬 시각화 라이브러리

01 데이터 분석을 위한 Elements, Widgets

✔ Chart Elements - Streamlit x Folium

streamlit app

draw support

dynamic map vs rerender

dynamic updates

geojson popup

limit data return

misc examples

responsive

simple popup

static map

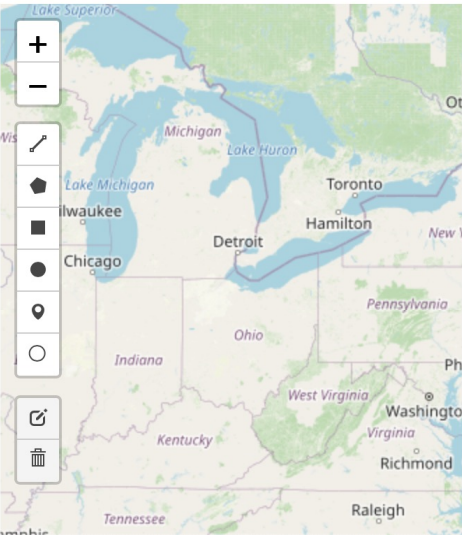
vector grid

streamlit-folium: Draw Support

Folium supports some of the [most popular leaflet plugins](#). In this example, we can add the [Draw](#) plugin to our map, which allows for drawing geometric shapes on the map.

When a shape is drawn on the map, the coordinates that represent that shape are passed back as a geojson feature via the `all_drawings` and `last_active_drawing` data fields.

Draw something below to see the return value back to Streamlit!



```
{
  "last_clicked": NULL,
  "last_object_clicked": NULL,
  "last_object_clicked_tooltip": NULL,
  "all_drawings": NULL,
  "last_active_drawing": NULL,
  "bounds": {
    "_southWest": {
      "lat": 31.015278981711266,
      "lng": -90.52734375
    },
    "_northEast": {
      "lat": 47.84265762816538,
      "lng": -50.75565566666667
    }
  }
}
```

streamlit app

draw support

dynamic map vs rerender

dynamic updates

geojson popup

limit data return

misc examples

responsive

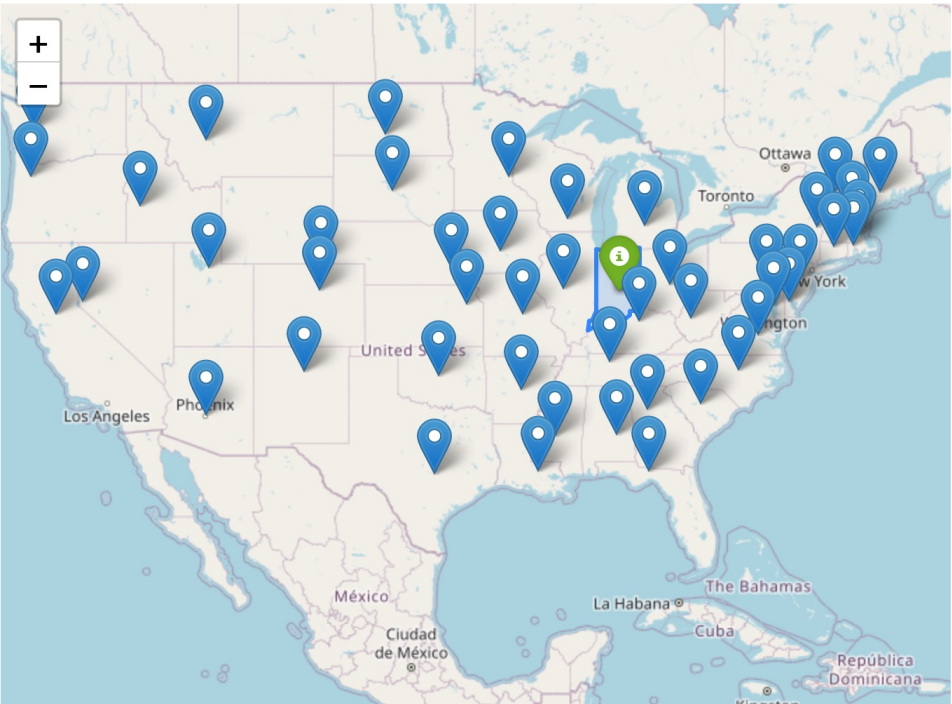
simple popup

static map

vector grid

Indiana

Population: 6,696,893



Folium 라이브러리를 사용한 지도 시각화

용어 해설

Folium

파이썬에서 지도를 다루는 대표적인 라이브러리

02

[실습] Data Elements와 여러가지 테이블

02 [실습] Data Elements와 여러가지 테이블

☑ Dataframe

```
import streamlit as st
import pandas as pd

df = pd.read_csv("avocado.csv")
st.dataframe(df)
```

	date	average_price	total_volume	4046	4225	4770	total_bags	small_bags
0	2015-01-04	1.22	40,873.28	2,819.5	28,287.42	49.9	9,716.46	9,186.9
1	2015-01-04	1.79	1,373.95	57.42	153.88	0	1,162.65	1,162.6
2	2015-01-04	1	435,021.49	364,302.39	23,821.16	82.15	46,815.79	16,707.1
3	2015-01-04	1.76	3,846.69	1,500.15	938.35	0	1,408.19	1,071.3
4	2015-01-04	1.08	788,025.06	53,987.31	552,906.04	39,995.03	141,136.68	137,146.0
5	2015-01-04	1.29	19,137.28	8,040.64	6,557.47	657.48	3,881.69	3,881.6
6	2015-01-04	1.01	80,034.32	44,562.12	24,964.23	2,752.35	7,755.62	6,064
7	2015-01-04	1.64	1,505.12	1.27	1,129.5	0	374.35	186.6
8	2015-01-04	1.02	491,738	7,193.87	396,752.18	128.82	87,663.13	87,406.8
9	2015-01-04	1.83	2,192.13	8.66	939.43	0	1,244.04	1,244.0

02 [실습] Data Elements와 여러가지 테이블

☑ Data editor

```
import streamlit as st
import pandas as pd

df = pd.read_csv("avocado.csv")
new_df = st.data_editor(df)
```

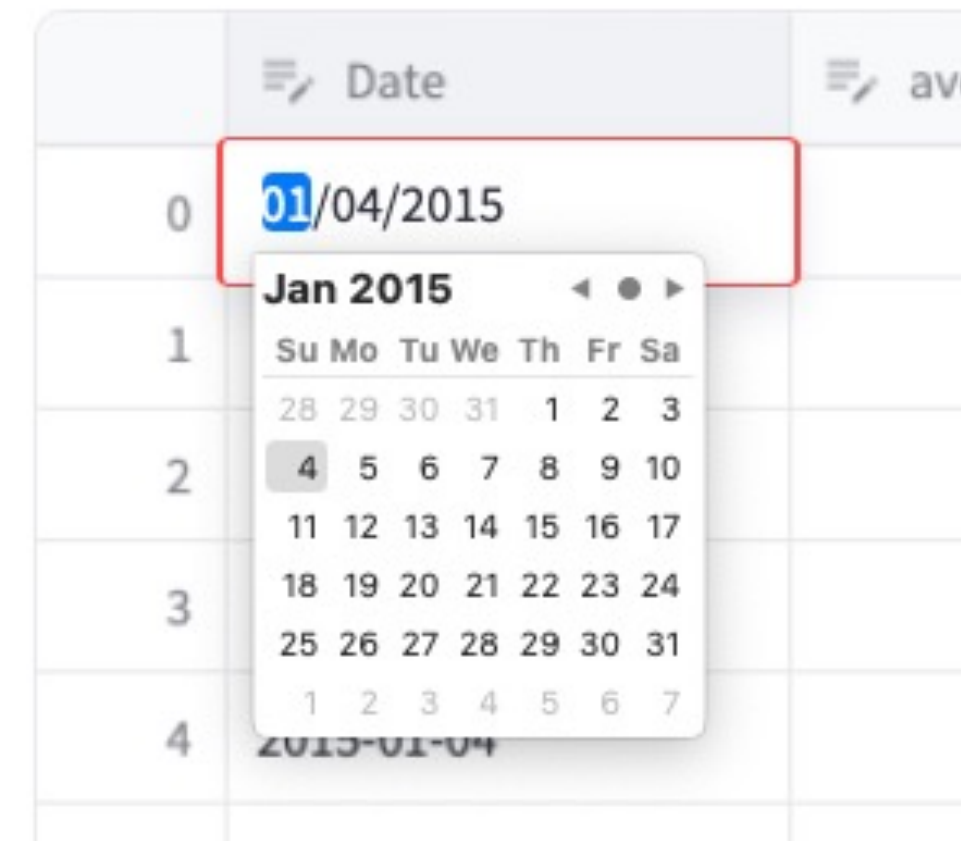
	☒ date	☒ average_price	☒ total_volume	☒ 4046	☒ 4225	☒ 4770	☒ total_bag
0	2015-01-04	1,000,000,000	40,873.28	2,819.5	28,287.42	49.9	9,716.4
1	2015-01-04	1.79	1,373.95	57.42	153.88	0	1,162.6
2	2015-01-04	1	435,021.49	364,302.39	23,821.16	82.15	46,815.7
3	2015-01-04	1.76	3,846.69	1,500.15	938.35	0	1,408.1
4	2015-01-04	1.08	788,025.06	53,987.31	552,906.04	39,995.03	141,136.6
5	2015-01-04	1.29	19,137.28	8,040.64	6,557.47	657.48	3,881.6
6	2015-01-04	1.01	80,034.32	44,562.12	24,964.23	2,752.35	7,755.6
7	2015-01-04	1.64	1,505.12	1.27	1,129.5	0	374.3
8	2015-01-04	1.02	491,738	7,193.87	396,752.18	128.82	87,663.1
9	2015-01-04	1.83	2,192.13	8.66	939.43	0	1,244.0

02 [실습] Data Elements와 여러가지 테이블

☑ Column config

```
df = pd.read_csv("avocado.csv")
df['date'] = df['date'].apply(lambda x: date.fromisoformat(x))

new_df = st.data_editor(df,
                        column_config={
                            "date": st.column_config.DateColumn("Date")
                        })
```



The screenshot shows a Streamlit data editor interface. The column header is 'Date'. The first row (index 0) contains the date '01/04/2015'. A calendar picker is open, showing the month of January 2015. The calendar has a grid with days of the week (Su, Mo, Tu, We, Th, Fr, Sa) and dates (1-31). The date '01' is highlighted. The second row (index 1) shows the date '2015-01-04'.

	Date	av
0	01/04/2015	
1		
2		
3		
4	2015-01-04	

02 [실습] Data Elements와 여러가지 테이블

☑ Column config

```
plot_table = {
    "geography": [
        "Boston",
        "California",
        "Los Angeles"
    ]
}

line_chart = []
for i in plot_table["geography"]:
    line_chart.append(list(df[df["geography"]==i]["average_price"]))

plot_table["line"] = line_chart

st.dataframe(plot_table,
              column_config={
                  "line": st.column_config.LineChartColumn(
                      "Average Price",
                      width="medium"
                  )
              })
```

geography	Average Price
Boston	
California	
Los Angeles	

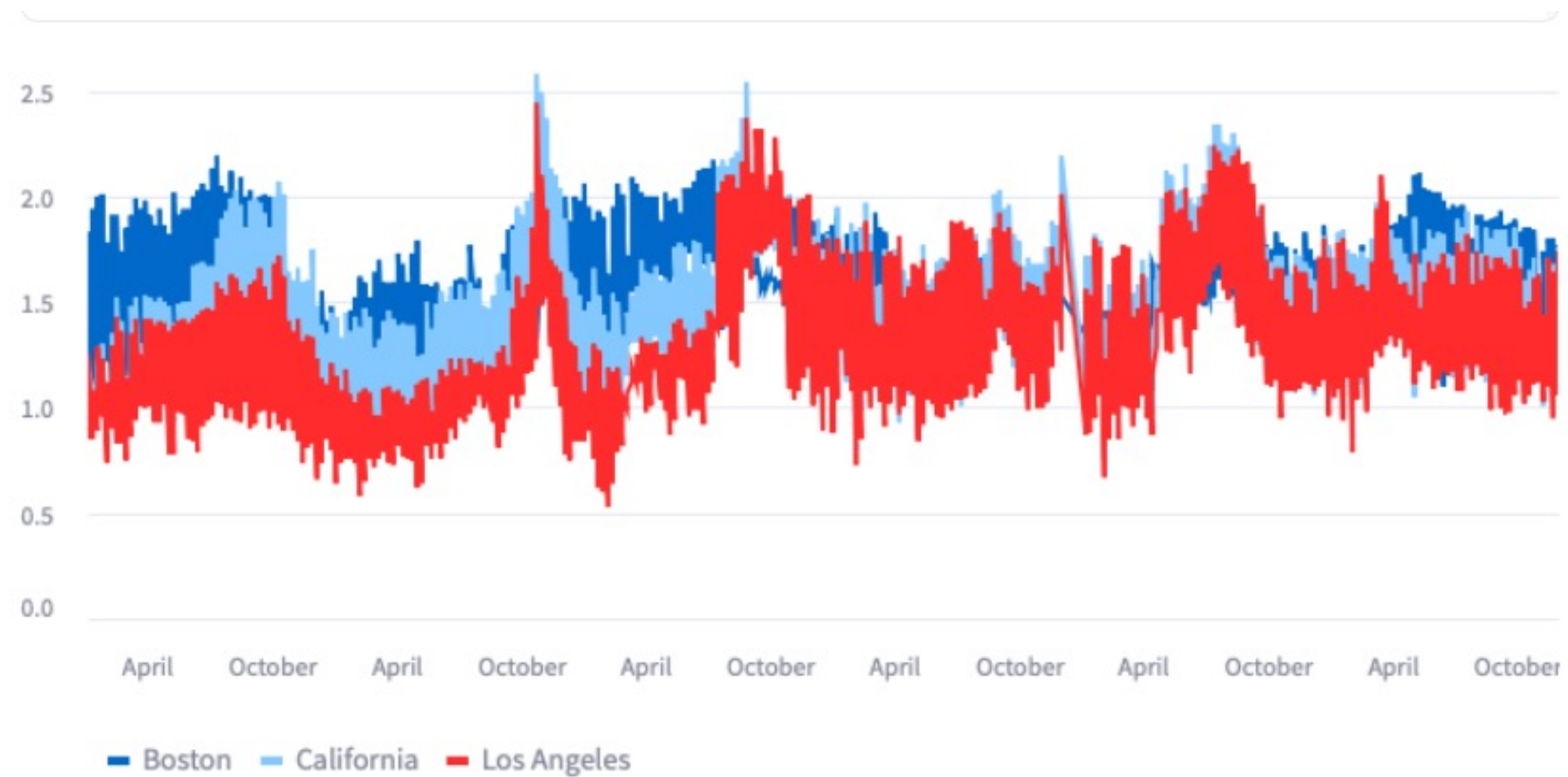
03

[실습] Chart Elements와 여러가지 시각화

03 [실습] Chart Elements와 여러가지 시각화

☑ Line chart

```
pivot_df = pd.pivot_table(df, index="date", columns=["geography", "type"])  
st.line_chart(pivot_df["average_price"][line_table["geography"]])
```



03 [실습] Chart Elements와 여러가지 시각화

☑ Bar chart

```
pivot_df = pd.pivot_table(df, index="date", columns=["geography", "type"])  
st.bar_chart(pivot_df["average_price"][pivot_table["geography"]])
```



03 [실습] Chart Elements와 여러가지 시각화

☑ Line chart with plotly

```
import plotly.express as px

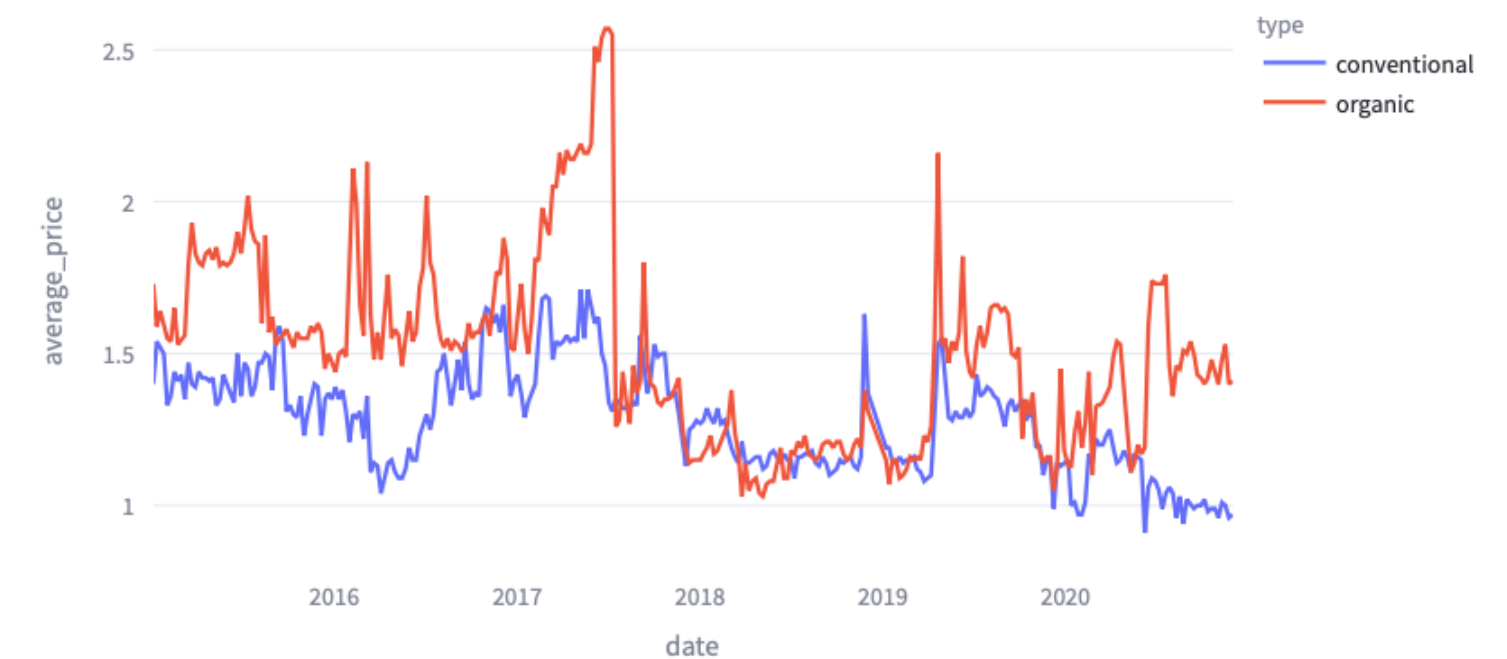
selected_geography = st.selectbox(label='Geography', options=df['geography'].unique())
submitted = st.button('Submit')
if submitted:
    filtered_avocado = df[df['geography'] == selected_geography]
    line_fig = px.line(filtered_avocado,
                       x='date', y='average_price',
                       color='type',
                       title=f'Avocado Prices in {selected_geography}')
    st.plotly_chart(line_fig)
```

Geography

Buffalo/Rochester

Submit

Avocado Prices in Buffalo/Rochester

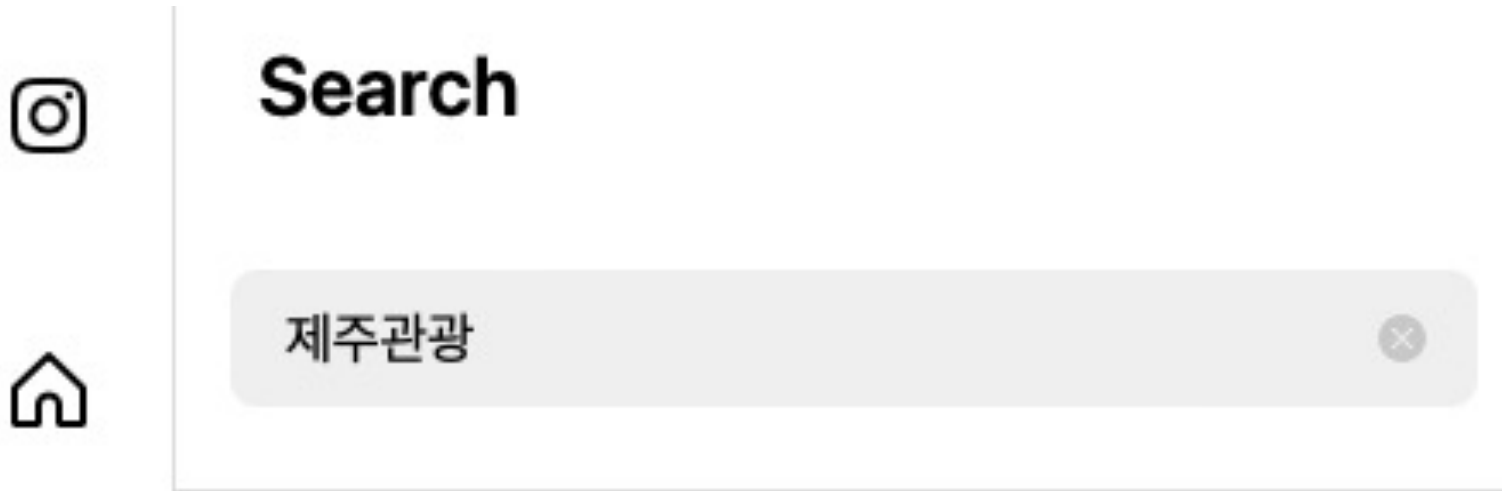


04

[실습] 지도 그리기

04 [실습] 지도 그리기

☑ 데이터 소개



인스타그램 제주도 관광지 크롤링 데이터

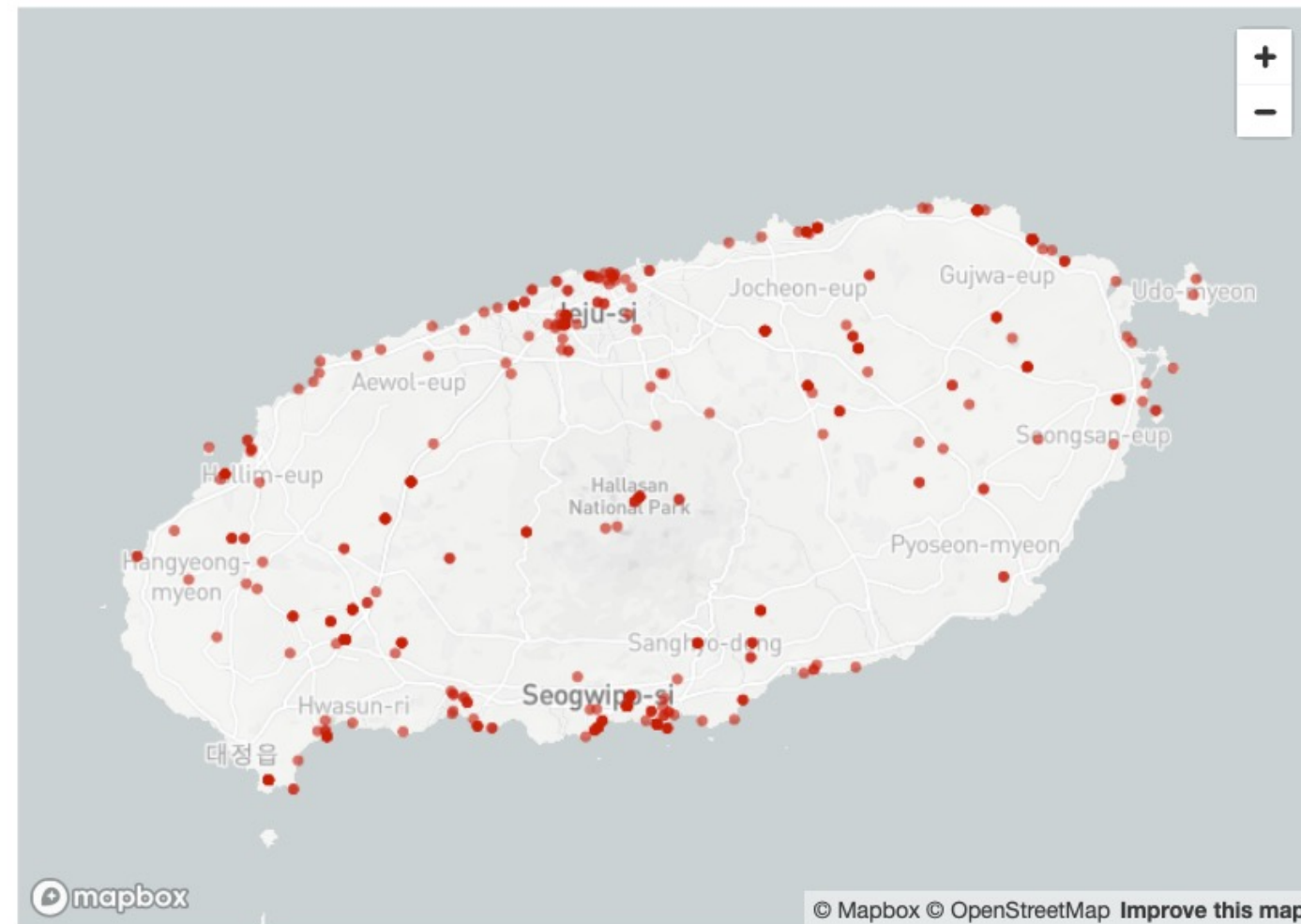
위치 값을 받아올 수 있는 데이터만 선별

	content	date	like	place	tags
0	#햄톨셀카 #햄토리제주내가 가서 좋았던곳 엄마미	11/14/17	90	엔트러사이트 제주 한림점	['#햄톨셀카', '#햄
1	#햄토리제주나 팔시려하는데 #양버터 정말 꿀깃ㅎ	11/15/17	90	채점석빵집	['#햄토리제주나',
2	#햄토리제주#채점석베이커리 에서 #빵 사들고 #바	11/15/17	132	Vadada 바다다	['#햄토리제주', '#
3	#햄토리제주함덕가면 먹어보라는 친구추천으로 #	11/17/17	96	오드랑베이커리	['#햄토리제주함덕
4	#햄토리제주#월정리로와 카페에서 #월정리해변 C	11/18/17	63	월정리해변	['#햄토리제주', '#
5	#햄토리제주#당근케이크 유명하다더니 정말 맛있	11/19/17	85	구좌상회작업실	['#햄토리제주', '#
6	#햄토리제주 #햄토리가시아방#돔베고기 정말정말	11/19/17	90	가시아방국수	['#햄토리제주', '#
7	빛#nimomevintagelounge #니모메빈티지라운지	7/7/18	46	니모메빈티지라운지	['#nimomevinta
8	휴애리 🍌 #에스타그램#48개월#곰하윤#매우귀이	10/9/18	20	휴애리 자연생활공원	['#에스타그램', '#
9	#서귀포 #올레시장 #매일올레시장 #제주시장 #제	10/9/18	30	서귀포 매일올레시장	['#서귀포', '#올레

04 [실습] 지도 그리기

☑ Map chart

```
st.title("제주도에서 가장 핫한 곳은?")  
  
df = pd.read_csv("jeju_place.csv")  
  
st.dataframe(df)  
  
df[["lat", "lon"]] = df[["위도", "경도"]]  
  
st.map(df)
```



04 [실습] 지도 그리기

☑ Map chart with folium

```
from streamlit_folium import folium_static
import folium
from folium.plugins import MarkerCluster

st.title("제주도에서 가장 핫한 곳은?")

df = pd.read_csv("jeju_place.csv")

m = folium.Map([sum(df["위도"])/len(df["위도"]),sum(df["경도"])/len(df["경도"])],zoom_start=9)
marker_cluster = MarkerCluster().add_to(m)

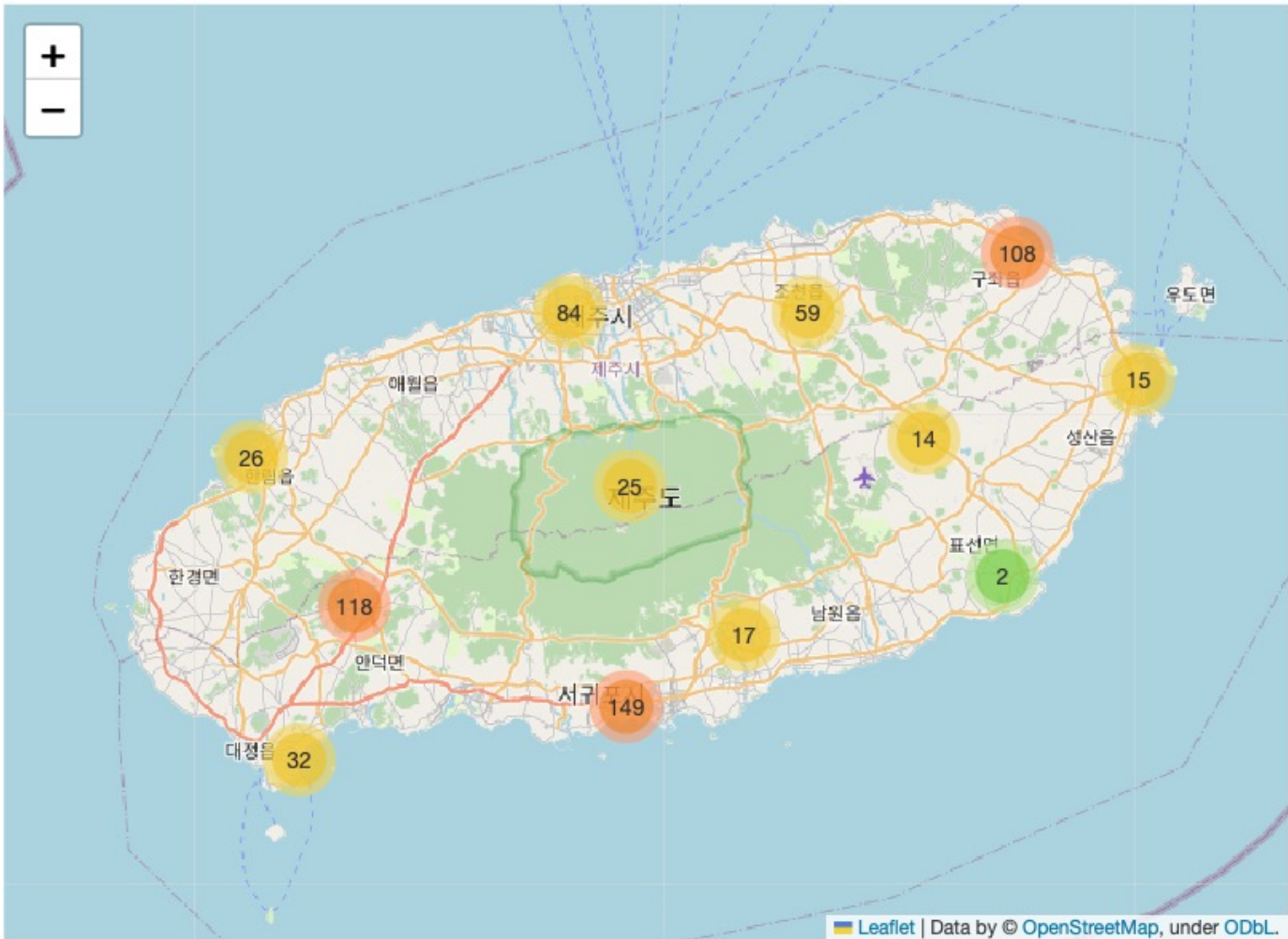
for i, j in zip(df["위도"],df["경도"]):
    folium.CircleMarker(
        [i,j]
    ).add_to(marker_cluster)

folium_static(m)
```


04 [실습] 지도 그리기

☑ Map chart with folium

제주도에서 가장 핫한 곳은?



제주도에서 가장 핫한 곳은?

